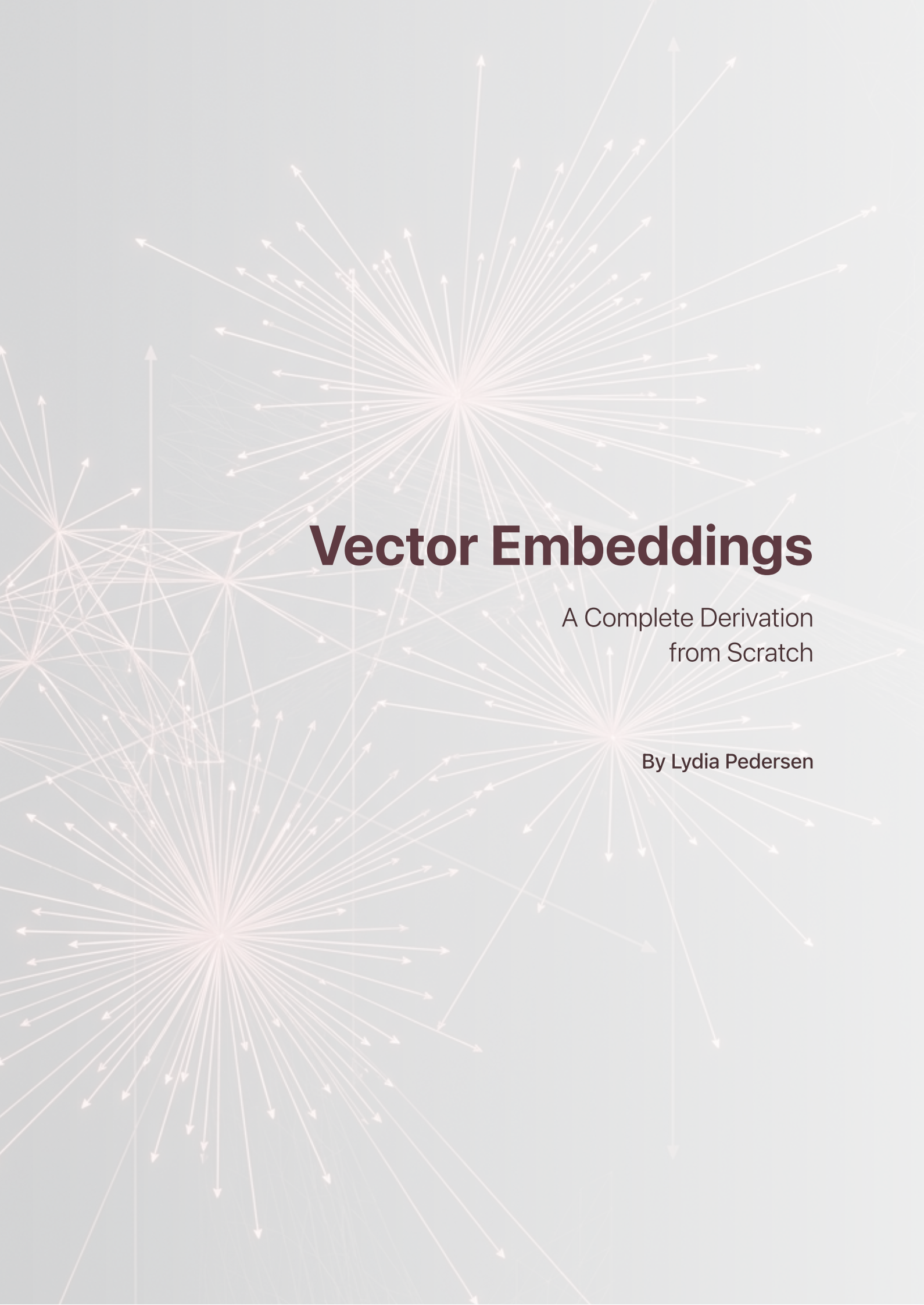


Vector Embeddings

A Complete Derivation
from Scratch

By Lydia Pedersen

Vector Embeddings: A Complete Derivation from Scratch

Copyright © 2026 Lydia Pedersen. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Table of Contents

Preface: The Gap Between Discrete and Continuous

Chapter 1: One-Hot Encoding and the Embedding Matrix

Chapter 2: The Continuous Vector Space

Chapter 3: The Forward Pass, Softmax, and Cross-Entropy Loss

Chapter 4: Backpropagation Through Loss and Softmax

Chapter 5: Backpropagation Through the Weight Matrices

Chapter 6: Convergent Geometry and Linear Substructures

Chapter 7: The Embedding Tensor and the Limits of Static Representations

Preface: The Gap Between Discrete and Continuous

The preceding Tokenization series established the mathematical process by which raw text is compressed into a sequence of discrete integer IDs. The subsequent Transformer series begins with a dense, continuous tensor of embedding vectors already loaded with semantic meaning. Between these two endpoints exists a structural gap: the mechanism by which discrete integers become continuous vectors whose geometric relationships encode the distributional structure of language.

This series derives that mechanism from first principles.

The Scope

The derivation proceeds through a shallow, two-layer neural network whose sole purpose is to train the embedding matrix $W_E \in \mathbb{R}^{V \times d_{\text{model}}}$. Every mathematical operation is computed explicitly using a concrete toy configuration:

- **Vocabulary Cardinality ($V = 8$):** Eight 0-indexed word tokens spanning indices `0` through `7`.
- **Embedding Dimensionality ($d_{\text{model}} = 3$):** Three continuous floating-point coordinates per token.

Token ID	Token	One-Hot Vector (`1 x 8`)	Embedding Matrix Row
`0`	`The`	`[1, 0, 0, 0, 0, 0, 0, 0]`	Row `0` of `W_E`
`1`	`quick`	`[0, 1, 0, 0, 0, 0, 0, 0]`	Row `1` of `W_E`
`2`	`brown`	`[0, 0, 1, 0, 0, 0, 0, 0]`	Row `2` of `W_E`
`3`	`fox`	`[0, 0, 0, 1, 0, 0, 0, 0]`	Row `3` of `W_E`
`4`	`jumps`	`[0, 0, 0, 0, 1, 0, 0, 0]`	Row `4` of `W_E`
`5`	`over`	`[0, 0, 0, 0, 0, 1, 0, 0]`	Row `5` of `W_E`
`6`	`lazy`	`[0, 0, 0, 0, 0, 0, 1, 0]`	Row `6` of `W_E`
`7`	`dog`	`[0, 0, 0, 0, 0, 0, 0, 1]`	Row `7` of `W_E`

This toy configuration is intentionally minimal. Eight words and three dimensions suffice to trace every matrix multiplication and partial derivative by hand, while remaining expressive enough to exhibit the geometric phenomena (orthogonality, convergence, and linear substructures) that govern trained embedding spaces at scale.

The Architecture

The series follows the complete lifecycle of a single training step, then traces the cumulative effect of billions of such steps:

1. **One-Hot Encoding and the Embedding Matrix:** Converts a discrete token ID into a sparse one-hot vector and multiplies it by W_E to extract a dense embedding row.
2. **The Continuous Vector Space:** Derives the high-dimensional geometric properties of randomly initialized embeddings, including isotropy, concentration of measure, and mutual orthogonality.
3. **The Forward Pass, Softmax, and Cross-Entropy Loss:** Projects the extracted embedding through an un-embedding matrix W_U to produce raw logits, normalizes those scores via Softmax, and quantifies prediction error via Cross-Entropy Loss.
4. **Backpropagation Through Loss and Softmax:** Traces the error gradient backward through the loss and Softmax layers to derive the unified $\hat{\mathbf{y}} - \mathbf{y}$ logit gradient from first principles.
5. **Backpropagation Through the Weight Matrices:** Propagates the logit gradient through W_U and W_E, proving that one-hot input sparsity restricts each weight update to a single embedding row.
6. **Convergent Geometry and Linear Substructures:** Examines the cumulative effect of millions of gradient updates, demonstrating how co-occurrence statistics produce semantic clustering and linear vector arithmetic.
7. **The Embedding Tensor and the Limits of Static Representations:** Generalizes single-token lookup to batched sequence tensors and diagnoses the structural limits of static embeddings (context-blindness and order-agnosticism) that motivate the Transformer architecture.

Every partial derivative is derived explicitly. Every abstract result is grounded in the toy vocabulary. The mathematics speaks for itself.

Chapter 1: One-Hot Encoding and the Embedding Matrix

The preceding Tokenization series established the mathematical process of transforming variable, unstructured text into discrete integer sequences. Subword compression algorithms such as Byte Pair Encoding parse raw text into a finite, fixed-size vocabulary where every entry maps to a unique integer index.

Applying this vocabulary lookup to the 8-word toy corpus introduced in the Preface (`The` `quick` `brown` `fox` `jumps` `over` `lazy` `dog`, where $V = 8$) maps each word to a 0-indexed integer identifier from `0` through `7`. Extracting two adjectives from that vocabulary yields two discrete integer IDs:

`brown` (ID: 2) `lazy` (ID: 6)

The tokenization pipeline terminates at these integer assignments. Passing the raw integers `2` and `6` directly into a neural network creates an immediate mathematical contradiction.

The Problem with Discrete Integers

Deep learning models operate through continuous mathematics: matrix multiplication, calculus, and gradient descent. Discrete integer IDs function purely as categorical lookup keys and possess no intrinsic algebraic magnitude or metric distance.

Feeding the integer IDs `2` and `6` directly into a linear layer forces the network to treat arbitrary dictionary indices as continuous scalar magnitudes. The arithmetic implies that `lazy` (`6`) is three times as large as `brown` (`2`), and that the numerical distance between `brown` (`2`) and `lazy` (`6`) equals `4`. That numerical ordering is an artifact of dictionary assignment rather than a linguistic relationship; the integer `2` does not make the color brown algebraically smaller than the adjective lazy.

Participating in gradient-based optimization requires transforming these discrete categorical indices into continuous, dense vectors where every coordinate serves as an independently tunable parameter.

The One-Hot Encoding

The first step in this transformation converts each integer token ID into a mathematically neutral representation: a **one-hot encoded vector** $\mathbf{x} \in \{0, 1\}^{1 \times V}$.

A vocabulary of size $V = 8$ represents the token `brown` (index `2` under 0-based indexing) as a row vector of length 8 containing zeros at every position except index `2`, which holds a single `1`:

$$\mathbf{x}_{\text{brown}} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{1 \times 8}$$

This sparse basis vector encodes the categorical identity of the token without introducing false numerical ordering across vocabulary entries. The `1` at index `2` marks the presence of `brown`, while the `0` entries at indices `0, 1, 3, 4, 5, 6, 7` mark the absence of every other vocabulary token. A one-hot vector remains completely sparse and fixed, providing no tunable parameters to capture semantic similarity. Encoding continuous relationships requires projecting this sparse vector into a learnable coordinate space.

The Embedding Matrix (W_E)

Projecting discrete tokens into a continuous vector space requires initializing a two-dimensional parameter grid called the **embedding matrix**, denoted as $W_E \in \mathbb{R}^{V \times d_{\text{model}}}$.

The matrix W_E operates as a parameterized lookup table. The matrix allocates one row for each of the V tokens in the vocabulary, and the width of each row equals the model hidden dimensionality d_{model} .

The toy configuration with vocabulary size $V = 8$ and embedding dimension $d_{\text{model}} = 3$ instantiates an 8×3 embedding matrix containing $8 \times 3 = 24$ learnable parameters (whereas a production architecture with $V = 50,000$ and $d_{\text{model}} = 512$ instantiates $50,000 \times 512 = 25,600,000$ parameters). At initialization, every entry $w_{i,m}$ (representing vocabulary row $i \in \{0, \dots, V-1\}$ and embedding dimension $m \in \{1, \dots, d_{\text{model}}\}$) is sampled as a random floating-point number.

The Mathematical Extraction

Standard vector-matrix multiplication maps the sparse one-hot row vector $\mathbf{x}_{\text{brown}} \in \mathbb{R}^{1 \times 8}$ to its dense embedding vector $\mathbf{h} \in \mathbb{R}^{1 \times 3}$.

Multiplying the one-hot vector $\mathbf{x}_{\text{brown}}$ by the embedding matrix W_E scales every row $i \neq 2$ by 0 and scales row 2 by 1 , isolating the third row of the matrix:

$$\mathbf{h} = \mathbf{x}_{\text{brown}} \times W_E = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{1\times 8} \times \begin{bmatrix} w_{0,1} & w_{0,2} & w_{0,3} \\ w_{1,1} & w_{1,2} & w_{1,3} \\ \mathbf{w_{2,1}} & \mathbf{w_{2,2}} & \mathbf{w_{2,3}} \\ w_{3,1} & w_{3,2} & w_{3,3} \\ \vdots & \vdots & \vdots \\ w_{7,1} & w_{7,2} & w_{7,3} \end{bmatrix}_{8\times 3}$$

$$\mathbf{h} = \begin{bmatrix} w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix}_{1\times 3} = \begin{bmatrix} -0.3 & 0.9 & 0.4 \end{bmatrix}_{1\times 3}$$

Here $\mathbf{h}$ denotes the extracted 1×3 hidden embedding vector for `brown`, populated with the concrete initial weights $\begin{bmatrix} -0.3 & 0.9 & 0.4 \end{bmatrix}$ traced throughout Chapters 3 through 7. This vector-matrix multiplication performs a linear row-selection operation: the one-hot encoding plucks row `2` from W_E , mapping the discrete integer ID `2` into a continuous vector of tunable parameters.

The projection through W_E bridges the boundary between discrete token indices and continuous vector geometry. The next chapter examines the high-dimensional geometric structure of these randomly initialized vectors and the mathematical consequences of scattering tokens across a high-dimensional space.

Chapter 2: The Continuous Vector Space

The linear projection of a one-hot vector through the embedding matrix W_E alters the mathematical domain of the tokens, bridging the gap from a discrete categorical set ($\mathbb{Z}$) into a dense, continuous vector space ($\mathbb{R}^{d_{\text{model}}}$).

This domain transition is a structural prerequisite for deep learning.

The Differentiable Manifold

Neural networks learn through calculus by computing partial derivatives and traversing a continuous loss landscape via backpropagation. Differential calculus requires a continuous manifold: a discrete integer ID admits no derivative, and a categorical label cannot absorb a microscopic numerical perturbation.

Projecting tokens into $\mathbb{R}^{d_{\text{model}}}$ parameterizes each vocabulary entry as a vector of floating-point coordinates. Scaling the hidden dimension from the 3-dimensional whiteboard model ($d_{\text{model}} = 3$) to a production dimensionality of $d_{\text{model}} = 512$ represents a token such as `brown` with 512 independent, tunable parameters:

$$\mathbf{v}_{\text{brown}} = [0.124 \quad -0.841 \quad 0.339 \quad \dots \quad -0.052] \in \mathbb{R}^{512}$$

Continuous coordinates impose no discrete grid constraints on parameter updates. A single gradient descent step can apply an arbitrarily small perturbation to $\mathbf{v}_{\text{brown}}$ (such as subtracting 0.001 from its first coordinate) to reduce the training loss incrementally. Repeated continuous perturbations allow the optimization process to translate statistical co-occurrence patterns into geometric proximity.

The Isotropic Expanse

These initial 512 dimensions do not correspond to latent human concepts. At initialization, one dimension does not encode plurality while another encodes sentiment.

Model initialization populates the entries of W_E by sampling independently from a zero-mean probability distribution, such as a Gaussian distribution $\mathcal{N}(0, \sigma^2)$ with mean 0 and variance σ^2 . Independent sampling from an identical symmetric distribution across all coordinates produces an **isotropic** vector space, meaning its statistical properties remain uniform in every direction.

High-dimensional geometry invalidates low-dimensional intuition about random point clouds. In a 512-dimensional space, a statistical phenomenon known as **concentration of measure** governs vector magnitudes. Computing the Euclidean length (norm) of a vector $\mathbf{v} \in \mathbb{R}^d$ requires summing its $d = 512$ squared coordinates and taking the square root:

$$\|\mathbf{v}\| = \sqrt{\sum_{m=1}^{d} v_m^2}$$

Summing 512 independent random variables invokes the Law of Large Numbers, causing the Euclidean norms of all randomly initialized token vectors to concentrate tightly around $\sqrt{d \cdot \sigma^2}$. Rather than forming an uneven solid cloud, random initialization places every token vector onto a thin spherical shell of radius $\sqrt{d}\sigma$, devoid of directional bias.

The exponential growth of volume across dimensions reinforces this isolation. A two-dimensional plane contains $2^2 = 4$ quadrants, so scattering a 50,000-token vocabulary across $\mathbb{R}^2$ forces thousands of tokens into the same quadrant.

A 512-dimensional space partitions into 2^{512} distinct sign regions, known mathematically as **orthants**. That exponent evaluates to $2^{512} \approx 1.34 \times 10^{154}$ orthants. Random sign assignment across 512 coordinates drops each initialized token into one of those 1.34×10^{154} orthants, ensuring that every vocabulary entry begins in extreme spatial isolation with negligible probability of accidental clustering.

The Geometry of Orthogonality

Within this isotropic expanse, a second high-dimensional invariant emerges: any two independently initialized token vectors (such as `brown` and `fox`) are nearly **orthogonal** (perpendicular) to one another.

Quantifying the directional alignment between two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ requires computing their **cosine similarity**, defined as the cosine of the angle θ between them. The geometric dot product relates the two vectors through the identity $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\| \cos(\theta)$, where $\|\mathbf{a}\|$ and $\|\mathbf{b}\|$ denote the Euclidean norms of $\mathbf{a}$ and $\mathbf{b}$. Dividing both sides by the product of the norms isolates $\cos(\theta)$:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \frac{\sum_{m=1}^{d} a_m b_m}{\|\mathbf{a}\|\|\mathbf{b}\|}$$

This identity demonstrates that the cosine of the angle θ is directly proportional to the algebraic **dot product**: the sum of the element-wise products $a_m b_m$ across all d coordinates.

Multiplying two coordinates a_m and b_m drawn independently from a symmetric distribution centered at zero produces a positive product when their signs match and a negative product with equal probability ($p = 0.5$) when their signs oppose.

In a 3-dimensional toy space ($d = 3$), the dot product sums only three random products. Small sample sizes exhibit high variance: all three terms share the same sign with probability $2 \times (0.5)^3 = 0.25$, producing wide fluctuations in $\cos(\theta)$ that frequently align random vectors in similar directions.

In a 512-dimensional space ($d = 512$), the dot product sums 512 independent zero-mean products. Law of Large Numbers convergence shrinks the relative variance of the normalized dot product at a rate of $1/\sqrt{d}$:

- **Sign Balance:** Expected sign symmetry yields roughly 256 positive products ($a_m b_m > 0$) and 256 negative products ($a_m b_m < 0$).
- **Magnitude Symmetry:** Symmetric sampling around zero ensures that the positive terms and negative terms share the same expected magnitude.
- **Mutual Cancellation:** Summing all 512 terms cancels the positive half against the negative half, driving the expected dot product $\mathbf{a} \cdot \mathbf{b}$ and the resulting cosine similarity $\cos(\theta)$ toward 0.0 (with standard deviation $1/\sqrt{512} \approx 0.044$).

Spatial Regime	Dimensionality (`d`)	Distinct Orthants (`2^d`)	Dot-Product Terms	Pairwise Cosine Similarity
2D Plane	`2`	`4`	`2`	High variance (`0.71` std dev); severe crowding
3D Whiteboard Model	`3`	`8`	`3`	Moderate variance (`0.58` std dev); frequent alignment
Production Space	`512`	`2^512` (`~1.34e154`)	`512`	Tight concentration near `0.0` (`0.044` std dev); near-orthogonal

An angle whose cosine equals 0.0 measures $90°$. Every pair of randomly initialized tokens in a high-dimensional embedding matrix begins geometrically orthogonal to one another.

A Feature, Not a Bug

Near-orthogonality at initialization provides a foundational structural advantage for gradient-based optimization.

Orthogonal vectors establish an unentangled coordinate baseline. Random initial clustering would force early training steps to expend gradient updates untangling spurious correlations between unrelated words. Initializing every token at roughly $90°$ from every other token presents the optimizer with an uncorrelated geometric slate, allowing backpropagation to pull co-occurring tokens together and push unrelated tokens apart purely in response to corpus statistics.

The initialized embedding matrix W_E thus begins as a nearly orthogonal set of points on a hypersphere, encoding neither semantic clusters nor grammatical hierarchy. Imposing linguistic structure onto this isotropic expanse requires evaluating predictions and updating the weights of W_E through the forward and backward passes derived in the next three chapters.

Chapter 3: The Forward Pass, Softmax, and Cross-Entropy Loss

Transforming a randomly initialized embedding matrix (W_E) into a structured semantic space requires an objective function that evaluates vector positions on every training step. A shallow neural network constructed around W_E provides this evaluation mechanism through **Next-Token Prediction**: observing a context token and predicting the most probable subsequent token in the corpus.

The Proxy of Prediction

The choice of next-token prediction requires justification. The training objective aims to capture semantic relationships across words, yet no supervised database of hand-labeled semantic attributes (such as `is_animal = 1.0`) exists to guide the updates.

The architecture relies instead on **Distributional Semantics**, formulated by linguist J.R. Firth as the principle that a word is characterized by the company it keeps. Observing that the nouns `fox`, `wolf`, and `dog` frequently precede the same verbs (`hunts`, `sleeps`, `runs`) forces gradient descent to push their embedding vectors toward the same region of the coordinate space. Semantic structure is never programmed directly; it emerges as a geometric byproduct of optimizing the next-token prediction objective.

The 2-Layer Embedding Network

Static embedding models train through a shallow neural network comprising two linear projection layers:

1. **Layer 1 (`W_E` , The Embedding Projection):** Multiplies the one-hot encoded input vector $\mathbf{x} \in \{0,1\}^{1\times V}$ (representing the current context token) by $W_E \in \mathbb{R}^{V\times d_{\text{model}}}$ to extract a dense, continuous hidden vector $\mathbf{h} \in \mathbb{R}^{1\times d_{\text{model}}}$.
2. **Layer 2 (`W_U` , The Un-embedding Projection):** Multiplies the dense hidden vector $\mathbf{h}$ by a second learnable weight matrix $W_U \in \mathbb{R}^{d_{\text{model}}\times V}$ to project the representation back into the V -dimensional vocabulary space, producing an unnormalized score (logit) for every candidate next token.

The Hidden Layer

The **hidden layer** of this network consists of the intermediate state $\mathbf{h} \in \mathbb{R}^{1\times d_{\text{model}}}$ between the two projections. The training loop exists solely to optimize the rows of Layer 1 (W_E). Completing the training run allows the system to discard Layer 2 (W_U) and export the rows of W_E as the static token embeddings.

The Activation Function

The architecture omits non-linear activation functions (such as ReLU or GELU) between Layer 1 and Layer 2.

That omission distinguishes a static Embedding Network from the Feed-Forward Networks (FFNs) inside a Transformer. A Transformer applies non-linear activations to model complex feature interactions, whereas an Embedding Network preserves a pure linear projection so that directional and additive relationships (such as cosine similarity and vector offsets) remain linear across the coordinate space.

Stacking two linear matrices (W_E and W_U) without an intervening non-linearity allows their product to collapse algebraically into a single matrix ($W_E \times W_U = W_{\text{combined}} \in \mathbb{R}^{V \times V}$). The architecture maintains W_E and W_U as separate factors so that the d_{model} -dimensional bottleneck state $\mathbf{h}$ can be intercepted between them as the compressed token embedding.

A Toy Mathematical Walkthrough

The 8-word toy configuration demonstrates the complete forward pass through explicit numerical calculation:

- **Vocabulary ($V = 8$):** `The` (`0`), `quick` (`1`), `brown` (`2`), `fox` (`3`), `jumps` (`4`), `over` (`5`), `lazy` (`6`), `dog` (`7`).
- **Embedding Dimension ($d_{\text{model}} = 3$):** Three continuous coordinates per token.

The input sequence is `The quick brown`, and the prediction target is the subsequent token `fox` (index `3`).

(Note: Reaching the end of a sentence sets the prediction target to a dedicated End-of-Sequence `<EOS>` control token. The vocabulary allocates a standard row in W_E for `<EOS>`, learning its geometric coordinates by observing sentence-termination patterns across the corpus.)

Step 1: The Input

The active context token, `brown` (vocabulary index `2`), is encoded as a 1×8 one-hot row vector $\mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{1 \times 8}$$

Step 2: Layer 1 (The Embedding Lookup)

The forward pass multiplies the one-hot input vector $\mathbf{x}$ by the randomly initialized 8×3 embedding matrix W_E :

$$W_E = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ \mathbf{-0.3} & \mathbf{0.9} & \mathbf{0.4} \\ 0.2 & -0.2 & 0.1 \\ \vdots & \vdots & \vdots \end{bmatrix}_{8 \times 3}$$

Zero entries at every position of $\mathbf{x}$ except index `2` cause the vector-matrix product $\mathbf{h} = \mathbf{x} \times W_E$ to isolate row `2` :

$$\mathbf{h} = \begin{bmatrix} -0.3 & 0.9 & 0.4 \end{bmatrix}_{1 \times 3}$$

This 3-dimensional row vector $\mathbf{h}$ forms the **hidden layer** state, representing the current untrained embedding for `brown`.

Step 3: Layer 2 (The Un-embedding Projection)

The second layer projects the hidden state $\mathbf{h} \in \mathbb{R}^{1\times 3}$ back into the 8-word vocabulary space via the un-embedding matrix $W_U \in \mathbb{R}^{3\times 8}$:

$$\mathbf{z} = \mathbf{h} \times W_U$$

Evaluating this product against the current weights of W_U yields the 1×8 logit vector $\mathbf{z}$:

$$\mathbf{z} = \begin{bmatrix} 1.2 & -0.5 & 0.3 & 2.1 & -1.8 & 0.7 & 0.0 & 0.4 \end{bmatrix}_{1\times 8}$$

The vector $\mathbf{z}$ contains the raw **logits** (z_0 through z_7), which are unbounded real-valued scores measuring how strongly the network favors each vocabulary word as the next token.

Step 4: The Softmax Function

Evaluating prediction accuracy requires converting the unbounded logits $\mathbf{z}$ into a valid probability distribution $\hat{\mathbf{y}}$ whose components are strictly positive and sum to 1.0 . The **Softmax** function performs this normalization across the 0-indexed vocabulary ($j \in \{0, \dots, V-1\}$):

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=0}^{V-1} e^{z_j}}$$

Here $\hat{y}_i$ denotes the predicted probability assigned to token i , z_i denotes that token's logit, $e \approx 2.718$ is the base of the natural exponential function, and the denominator sums the exponentiated logits e^{z_j} across all $V = 8$ tokens in the vocabulary.

Computing the Softmax probability for the target token `fox` (index `3`, where $z_3 = 2.1$) proceeds in three steps:

1. **Exponentiate the target logit z_3 :**

$$e^{2.1} \approx 8.16$$

2. **Exponentiate all eight logits and sum the results across the vocabulary:**

$$\begin{aligned} \sum_{j=0}^{7} e^{z_j} &= e^{1.2} + e^{-0.5} + e^{0.3} + e^{2.1} + e^{-1.8} + e^{0.7} + e^{0.0} + e^{0.4} \\ &\approx 3.32 + 0.60 + 1.35 + 8.16 + 0.16 + 2.01 + 1.0 + 1.49 = 18.09 \end{aligned}$$

3. **Divide the target exponential by the normalization sum:**

$$\hat{y}_3 = \frac{8.16}{18.09} \approx 0.45$$

The network assigns a 45% probability ($\hat{y}_3 = 0.45$) to `fox` appearing as the next word. Evaluating this ratio across all eight indices produces the full predicted probability vector $\hat{\mathbf{y}} \in \mathbb{R}^{1\times 8}$.

Step 5: The Cross-Entropy Loss

Random weight initialization means this 45% prediction reflects untrained parameter values rather than learned linguistic structure. Driving gradient updates requires a scalar objective function that quantifies the divergence between the predicted distribution $\hat{\mathbf{y}}$ and the ground-truth distribution $\mathbf{y}$.

The ground-truth label $\mathbf{y} \in \{0, 1\}^{1\times 8}$ specifies that `fox` (index `3`) is the observed next token, forming a one-hot vector with $y_3 = 1.0$ and $y_i = 0$ at every other index.

Cross-Entropy Loss measures the divergence between the predicted distribution $\hat{\mathbf{y}}$ and the target distribution $\mathbf{y}$ using two foundational constructs from information theory:

1. **The Natural Logarithm as Surprise ($\log(\hat{y}_i)$):** Applying the natural logarithm (base e) to a predicted probability $\hat{y}_i \in (0, 1]$ maps a raw probability into a continuous measure of surprisal. Assigning a 99% probability to the observed token yields near-zero surprise ($\log(0.99) \approx -0.01$), whereas assigning a 1% probability produces a steep penalty ($\log(0.01) \approx -4.61$).
2. **The Weighted Summation as Expected Value:** Multiplying each token's true probability y_i by its log-probability $\log(\hat{y}_i)$ and negating the sum across all V vocabulary entries computes the expected surprisal of the model under the ground-truth distribution:

$$L = -\sum_{i=0}^{V-1} y_i \log(\hat{y}_i)$$

Here L denotes the scalar Cross-Entropy Loss, $y_i \in \{0, 1\}$ denotes the ground-truth probability for token i , and $\log(\hat{y}_i)$ denotes the natural logarithm of the model's predicted probability for token i .

The one-hot structure of the target distribution $\mathbf{y}$ simplifies this summation algebraically. Every incorrect vocabulary entry carries a ground-truth weight of $y_i = 0$, annihilating seven of the eight terms in the sum and leaving only the target index where $y_{\text{true}} = 1.0$.

Dropping the seven zeroed terms reduces the full summation to a single expression known as **Negative Log-Likelihood**:

$$L = -\log(\hat{y}_{\text{true}})$$

Each component of the Negative Log-Likelihood expression performs a distinct role:

- **Likelihood ($\hat{y}_{\text{true}}$):** Extracts the predicted probability assigned to the ground-truth token (0.45 for `fox`), which the training objective seeks to maximize toward 1.0 .
- **Natural Logarithm ($\log$):** Maps a probability on $(0, 1)$ to a negative real number ($\log(0.45) \approx -0.79$), penalizing low-confidence predictions non-linearly.
- **Negation ($-$):** Multiplies the negative log-probability by -1 to produce a positive loss scalar $L \geq 0$ that approaches 0 as confidence approaches 1.0 .

Substituting the predicted probability $\hat{y}_3 = 0.45$ for `fox` yields the loss for this forward pass:

$$L = -\log(0.45) \approx 0.79$$

This scalar $L = 0.79$ quantifies the total prediction error of the forward pass, providing the starting point for the backward pass derived in Chapter 4.

The Limit of Static Embeddings: Superposition

A structural limitation of a static embedding table is **polysemy**: a single word form carrying multiple distinct meanings.

The static lookup matrix W_E allocates only one row vector for the token `apple`. Corpus contexts such as *"The orchard harvested a ripe apple"* pull that single row vector toward agricultural and fruit coordinates, whereas technology contexts such as *"Apple released a new laptop"* pull the same row vector toward consumer electronics coordinates.

Across billions of training updates, the single vector for `apple` converges to a state of **superposition**, resting at the frequency-weighted center of mass between its competing senses.

Resolving that contextual ambiguity motivates the **Self-Attention** mechanism in the Transformer architecture. Self-Attention enables the static `apple` vector to attend to surrounding tokens in the active sequence and dynamically shift its representation toward either the fruit subspace or the technology subspace before the final prediction layer.

Before examining sequence-level contextualization in Chapter 7, Chapters 4 and 5 derive the backpropagation equations that physically adjust the weights of W_U and W_E via gradient descent.

Chapter 4: Backpropagation Through Loss and Softmax

The previous chapter ended with a single scalar: $L = 0.79$. This number represents the network's total error, a mathematical verdict on the quality of the forward pass. To learn, the network must translate this single scalar into targeted corrections across the individual weights of both matrices (W_U and W_E).

Backpropagation accomplishes this credit assignment through the systematic application of the calculus chain rule, working backward from the loss through each layer of the network. The output is the **gradient**, a collection of partial derivatives that specifies how much each weight contributed to the error and which direction reduces the loss.

From Scalar to Signal

The partial derivative $\frac{\partial L}{\partial w}$ answers a specific question: *holding every other weight in the network frozen, how does nudging this single weight w by an infinitesimally small amount change the loss L ?*

The optimization objective drives the loss toward zero, corresponding to an accurate prediction. A positive derivative indicates that increasing w increases the loss. A negative derivative indicates that increasing w decreases the loss. A zero derivative indicates that the weight sits at a stationary point where the loss is locally minimized. The magnitude measures how *sensitive* the loss is to this particular weight: a large magnitude means small changes to w cause large swings in the loss, whereas a magnitude near zero means the weight barely affects the prediction.

Computing this derivative for a weight in the final layer is direct because that layer connects immediately to the loss. For a weight in W_E , located at the first layer, the relationship is indirect: a change in W_E alters the hidden state $\mathbf{h}$, which alters the logits $\mathbf{z}$, which alters the Softmax probabilities $\hat{\mathbf{y}}$, which finally alters the loss L . The **chain rule** decomposes this dependency into a product of local derivatives:

$$\frac{\partial L}{\partial W_E} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial W_E}$$

Each factor in this product is a tractable local computation. The derivation proceeds step by step from the loss backward toward W_E .

The Loss Derivative: $\frac{\partial L}{\partial \hat{\mathbf{y}}}$

The first link in the chain measures how the loss changes when the predicted probabilities change.

Recall from Chapter 3 that the Cross-Entropy Loss, after collapsing under the one-hot encoded ground-truth vector, reduces to:

$$L = -\log(\hat{y}_{\text{true}})$$

The loss depends on a single element of the predicted distribution: $\hat{y}_{\text{true}}$, the probability the network assigned to the target word (`fox` , at index `3`). The loss expression is independent of the probabilities assigned to every other word, so the partial derivative of the loss with respect to any incorrect word's probability equals zero:

$$\frac{\partial L}{\partial \hat{y}_k} = 0 \quad \text{for all } k \neq \text{true}$$

For the target word, differentiation requires the derivative of $-\log(x)$ with respect to x . By standard calculus, the derivative of the natural logarithm $\log(x)$ is $\frac{1}{x}$, and the leading negative sign carries through:

$$\frac{\partial L}{\partial \hat{y}_{\text{true}}} = \frac{-1}{\hat{y}_{\text{true}}}$$

This inverse relationship calibrates the strength of the learning signal. A high predicted probability for the target word (such as $\hat{y}_{\text{true}} = 0.95$) produces a small derivative: $\frac{-1}{0.95} \approx -1.05$. The loss is barely sensitive to further changes because the prediction is already nearly accurate. A tiny predicted probability (such as $\hat{y}_{\text{true}} = 0.01$) produces a massive derivative: $\frac{-1}{0.01} = -100$. That steep slope generates a strong corrective signal, and the negative sign indicates that *increasing* $\hat{y}_{\text{true}}$ *decreases* the loss.

The Softmax Derivative: $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}}$

The second link in the chain measures how each Softmax output $\hat{y}_i$ changes when a single logit z_j is adjusted.

Three 0-based index variables ($i, j, k \in \{0, \ldots, V-1\}$) govern this derivation:

- **Output Index (i):** Identifies the vocabulary word whose **predicted probability** $\hat{y}_i$ is being measured.
- **Input Index (j):** Identifies the vocabulary word whose **logit** z_j is being perturbed.
- **Summation Index (k):** Ranges across all V words in the vocabulary inside the normalization denominator $\sum_{k=0}^{V-1} e^{z_k}$.

Recall the Softmax definition from Chapter 3:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{k=0}^{V-1} e^{z_k}}$$

The Softmax expression forms a fraction with numerator e^{z_i} and denominator $\sum_{k=0}^{V-1} e^{z_k}$. The denominator sums the exponentiated logits across the entire vocabulary. Adjusting a single logit z_j alters both the probability $\hat{y}_j$ for that word and the shared denominator, which shifts *every other* probability $\hat{y}_i$ in the distribution.

Differentiating this ratio requires the **quotient rule** from calculus. For any differentiable ratio $\frac{f}{g}$, the quotient rule states:

$$\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f' \cdot g \; - \; f \cdot g'}{g^2}$$

where f' denotes the derivative of the numerator and g' denotes the derivative of the denominator.

The Softmax numerator is $f = e^{z_i}$ and the denominator is $g = \sum_{k=0}^{V-1} e^{z_k}$. Applying the quotient rule requires the derivatives f' and g' of the numerator and denominator with respect to the target logit z_j . The derivative of the numerator e^{z_i} depends on whether z_j is the *same* variable as z_i or a *different* variable.

Differentiating the probability of `fox` with respect to the logit for `fox` itself ($i = j$) means the numerator $e^{z_{\text{fox}}}$ contains the variable of differentiation, yielding a non-zero derivative. Differentiating the probability of `fox` with respect to the logit for `The` ($i \neq j$) means the numerator $e^{z_{\text{fox}}}$ does not contain z_{The} , so the numerator acts as a constant whose derivative is zero. This structural distinction splits the Softmax derivative into two cases.

Case 1: $i = j$ (Same Word)

In the diagonal case, the probability $\hat{y}_i$ is differentiated with respect to its *own* logit, z_i , measuring how the predicted probability of `fox` changes when the logit for `fox` is adjusted.

The derivatives of both the numerator and denominator with respect to z_i evaluate as follows:

- **Numerator Derivative (f'):** $f' = \frac{\partial}{\partial z_i} e^{z_i} = e^{z_i}$. The natural exponential function is its own derivative, and the numerator contains z_i , producing a non-zero result.
- **Denominator Derivative (g'):** $g' = \frac{\partial}{\partial z_i} \sum_{k=0}^{V-1} e^{z_k} = e^{z_i}$. The sum contains V terms ($e^{z_0}, e^{z_1}, \ldots, e^{z_{V-1}}$), but only the single term e^{z_i} depends on z_i . Every other term acts as a constant with respect to z_i and vanishes under differentiation.

Substituting f , g , f' , and g' into the quotient rule formula $\frac{f'g - fg'}{g^2}$ yields:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i} \cdot \sum_{k=0}^{V-1} e^{z_k} \; - \; e^{z_i} \cdot e^{z_i}}{\left(\sum_{k=0}^{V-1} e^{z_k}\right)^2}$$

Factoring e^{z_i} out of the numerator produces:

$$= \frac{e^{z_i}\left(\sum_{k=0}^{V-1} e^{z_k} \; - \; e^{z_i}\right)}{\left(\sum_{k=0}^{V-1} e^{z_k}\right)^2}$$

Splitting this single fraction into a product of two ratios isolates the Softmax terms:

$$= \frac{e^{z_i}}{\sum_{k=0}^{V-1} e^{z_k}} \cdot \frac{\sum_{k=0}^{V-1} e^{z_k} \; - \; e^{z_i}}{\sum_{k=0}^{V-1} e^{z_k}}$$

The first fraction equals $\hat{y}_i$ by definition, and the second fraction simplifies to $1 - \hat{y}_i$ (the full sum minus the i -th term, divided by the full sum), yielding the diagonal Softmax derivative:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

This quadratic expression peaks at $\hat{y}_i = 0.5$ (maximum uncertainty, where $\hat{y}_i(1 - \hat{y}_i) = 0.25$) and shrinks toward zero as $\hat{y}_i$ approaches 0 or 1 . Confident predictions near the boundaries are *insensitive* to small logit perturbations, whereas uncertain predictions near the midpoint respond strongly because Softmax **saturates** at its extremes.

Case 2: $i \neq j$ (Different Words)

In the off-diagonal case, the probability $\hat{y}_i$ is differentiated with respect to a *different* logit, z_j , measuring how the predicted probability of `The` changes when the logit for `fox` is adjusted.

The numerator and denominator derivatives with respect to z_j evaluate as follows:

- **Numerator Derivative (f'):** $f' = \frac{\partial}{\partial z_j} e^{z_i} = 0$. The numerator e^{z_i} does not contain z_j and acts as a constant with respect to z_j , so its derivative evaluates to zero.
- **Denominator Derivative (g'):** $g' = \frac{\partial}{\partial z_j} \sum_{k=0}^{V-1} e^{z_k} = e^{z_j}$. Only the single term e^{z_j} in the summation depends on z_j .

Substituting into the quotient rule formula $\frac{f'g - fg'}{g^2}$ yields:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{0 \cdot \sum_{k=0}^{V-1} e^{z_k} \; - \; e^{z_i} \cdot e^{z_j}}{\left(\sum_{k=0}^{V-1} e^{z_k}\right)^2}$$

The first term in the numerator vanishes, leaving:

$$= \frac{-e^{z_i} \cdot e^{z_j}}{\left(\sum_{k=0}^{V-1} e^{z_k}\right)^2}$$

Factoring the expression into a product of two ratios isolates the Softmax probabilities for indices i and j :

$$= -\frac{e^{z_i}}{\sum_{k=0}^{V-1} e^{z_k}} \cdot \frac{e^{z_j}}{\sum_{k=0}^{V-1} e^{z_k}} = -\hat{y}_i \cdot \hat{y}_j$$

Chapter 3 established that all Softmax outputs sum to 1.0 , fixing the total probability mass. Increasing the logit for `fox` (z_j) raises the probability of `fox` ($\hat{y}_j$), forcing every other word's probability to shrink so the distribution continues to sum to 1.0 .

The off-diagonal derivative $-\hat{y}_i \cdot \hat{y}_j$ quantifies each competing word's rate of probability loss. With $\hat{y}_{\text{fox}} = 0.45$ and $\hat{y}_{\text{The}} = 0.18$ from Chapter 3, increasing the logit for `fox` reduces the probability of `The` at an instantaneous rate of $-0.18 \times 0.45 = -0.081$. By contrast, `jumps` holds a much smaller probability of $\hat{y}_{\text{jumps}} = 0.009$ and loses mass at a rate of only $-0.009 \times 0.45 = -0.004$. Words holding larger probability mass surrender more probability; words near zero remain barely affected.

Summary of the Softmax Derivative

These two cases specify the complete Jacobian of the Softmax function. Case 1 governs how a word's probability responds to its own logit, while Case 2 governs how every competing word's probability responds. Combining these two cases with the loss derivative yields the gradient of the loss with respect to the logits, $\frac{\partial L}{\partial \mathbf{z}}$.

Combining: The Gradient at the Logits ($\frac{\partial L}{\partial \mathbf{z}}$)

The first two links of the chain have been derived independently. The loss derivative governs how the loss responds to changes in the predicted probabilities, and the Softmax derivative governs how the predicted probabilities respond to changes in the logits. Chaining these two factors sums across all V intermediate probabilities:

$$\frac{\partial L}{\partial z_i} = \sum_{k=0}^{V-1} \frac{\partial L}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_i}$$

This summation ranges over every word in the vocabulary because changing a single logit z_i affects every Softmax output $\hat{y}_k$. Step 1 established that $\frac{\partial L}{\partial \hat{y}_k} = 0$ for every $k \neq \text{true}$, so every term in the sum where k is an incorrect word multiplies by zero and vanishes. Only the single term where $k = \text{true}$ survives:

$$\frac{\partial L}{\partial z_i} = \frac{-1}{\hat{y}_{\text{true}}} \cdot \frac{\partial \hat{y}_{\text{true}}}{\partial z_i}$$

Substituting the appropriate Softmax derivative case depends on whether z_i is the logit for the target word or an incorrect word.

For the target class ($i = \text{true}$): Substituting Case 1, where $\frac{\partial \hat{y}_{\text{true}}}{\partial z_{\text{true}}} = \hat{y}_{\text{true}}(1 - \hat{y}_{\text{true}})$, yields:

$$\frac{\partial L}{\partial z_{\text{true}}} = \frac{-1}{\hat{y}_{\text{true}}} \cdot \hat{y}_{\text{true}}(1 - \hat{y}_{\text{true}})$$

Canceling $\hat{y}_{\text{true}}$ between the denominator and numerator simplifies the expression:

$$= -(1 - \hat{y}_{\text{true}}) = \hat{y}_{\text{true}} - 1$$

For every incorrect class ($i \neq \text{true}$): Substituting Case 2, where $\frac{\partial \hat{y}_{\text{true}}}{\partial z_i} = -\hat{y}_{\text{true}} \cdot \hat{y}_i$, yields:

$$\frac{\partial L}{\partial z_i} = \frac{-1}{\hat{y}_{\text{true}}} \cdot (-\hat{y}_{\text{true}} \cdot \hat{y}_i)$$

The two negative signs cancel each other, and $\hat{y}_{\text{true}}$ cancels between the numerator and denominator:

$$= \hat{y}_i$$

The Unified Gradient Expression

The two cases produce two apparent expressions:

- **Target Class ($i = \text{true}$, `fox` at index `3`):** The logit gradient evaluates to $\hat{y}_{\text{true}} - 1$.
- **Incorrect Classes ($i \neq \text{true}$, such as `The` , `quick` , `brown`):** The logit gradient evaluates to $\hat{y}_i$.

The one-hot target vector $\mathbf{y}$ unifies both formulas because $y_i = 1$ at index `3` (`fox`) and $y_i = 0$ at every other index. Expressing the gradient at every index i as $\hat{y}_i - y_i$ recovers both cases simultaneously:

Vocabulary Token	Index (`i`)	One-Hot Target (`y_i`)	Unified Formula	Evaluated Form	Toy Numerical Gradient
`The`	`0`	`y_0 = 0`	`y_hat_0 - 0`	`y_hat_0` (Incorrect class)	`+0.18`
`quick`	`1`	`y_1 = 0`	`y_hat_1 - 0`	`y_hat_1` (Incorrect class)	`+0.03`
`fox` (Target)	`3`	`y_3 = 1`	`y_hat_3 - 1`	`y_hat_true - 1` (Target class)	`0.45 - 1 = -0.55`

Both cases obey the identical algebraic rule: $\hat{y}_i - y_i$. The one-hot structure of $\mathbf{y}$ absorbs the case distinction because the -1 subtraction occurs solely at the target index where $y_i = 1$. Stacking all V indices yields a single vector equation for the gradient across the entire vocabulary:

$$\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$$

The entire derivation across quotient rules, logarithmic derivatives, and Softmax case-splitting collapses to **predicted minus truth**.

Cross-Entropy Loss is mathematically structured to pair with Softmax and produce this cancellation. The logarithm in the loss inverts the exponential in Softmax, and the normalization denominator keeps the resulting gradient bounded between -1 and $+1$. This algebraic pairing makes Cross-Entropy the standard loss function for classification, guaranteeing numerically stable gradients that point directly toward the target distribution.

In the 8-word toy network, this gradient vector is interpretable at a glance. Every incorrect word in the vocabulary receives a *positive* gradient ($\hat{y}_i - 0 = \hat{y}_i$), signaling that its logit should decrease under gradient descent. The target word `fox` receives a *negative* gradient ($\hat{y}_{\text{fox}} - 1 = -0.55$), signaling that its logit should increase. The magnitudes are self-calibrating: the most overconfident wrong predictions receive the largest positive gradients and the strongest downward corrections.

The gradient at the logits has been established. The next chapter continues the backward pass through the weight matrices.

Chapter 5: Backpropagation Through the Weight Matrices

The previous chapter established the gradient at the logits: $\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$. The backward pass now continues through the weight matrices to compute the gradient at the embedding layer itself.

Propagating Through Layer 2: $\frac{\partial \mathbf{z}}{\partial \mathbf{h}}$

The first two links of the chain have been combined to form $\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$. The next link is $\frac{\partial \mathbf{z}}{\partial \mathbf{h}}$, measuring how the logits change when the hidden state (the embedding vector for `brown`) is altered.

Recall from Chapter 3 that the logits are computed by multiplying the hidden state $\mathbf{h}$ by the un-embedding matrix W_U:

$$\mathbf{z} = \mathbf{h} \times W_U$$

The output $\mathbf{z}$ is a 1×8 row vector containing one logit per vocabulary word (`The`, `quick`, `brown`, `fox`, `jumps`, `over`, `lazy`, `dog`, indexed $j = 0, \ldots, 7$). In the toy model, $\mathbf{h}$ is a 1×3 vector (the 3-dimensional embedding for `brown`) and W_U is a 3×8 matrix. Each logit z_j equals the dot product of $\mathbf{h}$ with column j of W_U.

Differentiating this projection requires expanding the operation element-wise. The index $m \in \{1, 2, 3\}$ denotes a specific **embedding dimension** in the 3-dimensional toy model. Written out for a single logit z_j:

$$z_j = h_1 \cdot W_{U_{1,j}} + h_2 \cdot W_{U_{2,j}} + h_3 \cdot W_{U_{3,j}}$$

That dot product expands into a sum of three terms, each multiplying one embedding dimension h_m by its corresponding weight $W_{U_{m,j}}$. Differentiating this sum with respect to a specific embedding dimension h_m measures how z_j changes when h_m is adjusted.

The weights $W_{U_{1,j}}$, $W_{U_{2,j}}$, and $W_{U_{3,j}}$ act as fixed constants during this step. Each term in the sum takes the form of a constant multiplying h_m or a constant multiplying an independent coordinate. Differentiating with respect to h_m separates the terms into two groups:

- **Active Term ($h_m \cdot W_{U_{m,j}}$):** Contains h_m, and differentiating a variable multiplied by a constant leaves the constant factor $W_{U_{m,j}}$.
- **Remaining Terms ($h_n \cdot W_{U_{n,j}}$ for $n \neq m$):** Do not contain h_m, so they act as constants with respect to h_m and their derivatives evaluate to zero.

The entire sum collapses to a single surviving term:

$$\frac{\partial z_j}{\partial h_m} = W_{U_{m,j}}$$

The partial derivative equals the weight that multiplied h_m in the forward pass. That weight $W_{U_{m,j}}$ governs how much influence the m -th embedding dimension exerts on the j -th logit: a large weight means that dimension strongly affects that logit, whereas a weight near zero means it has almost no effect.

The Gradient at the Hidden State ($\frac{\partial L}{\partial \mathbf{h}}$)

Chaining this local derivative with $\frac{\partial L}{\partial \mathbf{z}}$ yields the gradient of the loss with respect to $\mathbf{h}$. A single embedding coordinate h_m participates in all 8 dot products across $\mathbf{z}$, so the chain rule sums its contributions across all 8 vocabulary indices ($j = 0, \ldots, 7$):

$$\frac{\partial L}{\partial h_m} = \sum_{j=0}^{7} \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial h_m} = \sum_{j=0}^{7} (\hat{y}_j - y_j) \cdot W_{U_{m,j}}$$

Each term in this sum isolates how much embedding dimension m of `brown` 's vector contributed to the error at vocabulary word j . The error at each word ($\hat{y}_j - y_j$) multiplies the weight connecting h_m to that word ($W_{U_{m,j}}$), summed across all 8 vocabulary words.

Computing this sum for all three embedding dimensions ($m = 1, 2, 3$) simultaneously yields the complete hidden-state gradient in matrix notation:

$$\frac{\partial L}{\partial \mathbf{h}} = (\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$$

The transpose W_U^T reverses the direction of the projection. During the forward pass, W_U projected the 3 -dimensional embedding $\mathbf{h}$ *up* into 8 -dimensional vocabulary space to produce the logits. During backpropagation, W_U^T projects the 8 -dimensional error signal *back down* into 3 -dimensional embedding space. The result is a 1×3 gradient vector that inhabits the same coordinate system as `brown` 's embedding vector, specifying the direction and magnitude needed to nudge `brown` 's vector to reduce the loss.

Updating W_U Along the Way

Before continuing backward to Layer 1, note that W_U itself contains learnable weights that require updating. Their gradients follow from the same element-wise expansion:

$$z_j = h_1 \cdot W_{U_{1,j}} + h_2 \cdot W_{U_{2,j}} + h_3 \cdot W_{U_{3,j}}$$

Differentiating this expression with respect to a specific *weight* $W_{U_{m,j}}$ rather than an embedding coordinate h_m reverses the roles of constants and variables: the embedding coordinates h_1, h_2, h_3 act as fixed constants computed during the forward pass, and $W_{U_{m,j}}$ acts as the variable of differentiation.

- **Active Term ($h_m \cdot W_{U_{m,j}}$):** Contains the target variable $W_{U_{m,j}}$, and differentiating a constant multiplied by a variable leaves the constant factor h_m .
- **Remaining Terms ($h_n \cdot W_{U_{n,j}}$ for $n \neq m$):** Do not contain $W_{U_{m,j}}$, so their derivatives evaluate to zero.

Only the active term survives differentiation:

$$\frac{\partial z_j}{\partial W_{U_{m,j}}} = h_m$$

Applying the chain rule connects this local derivative to the loss gradient:

$$\frac{\partial L}{\partial W_{U_{m,j}}} = \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{U_{m,j}}} = (\hat{y}_j - y_j) \cdot h_m$$

Each weight's gradient equals the product of two quantities: the error signal at the vocabulary word it connects to ($\hat{y}_j - y_j$) and the embedding coordinate value that flowed through it during the forward pass (h_m). In matrix form across all coordinates and vocabulary words, this outer product is $\frac{\partial L}{\partial W_U} = \mathbf{h}^T \cdot (\hat{\mathbf{y}} - \mathbf{y})$. These updates are applied alongside the W_E updates at the end of the step.

Propagating Through Layer 1: $\frac{\partial \mathbf{h}}{\partial W_E}$

The backward pass now reaches the first layer, which forms the origin of the forward pass and the final link in the derivative chain. In the forward pass, the hidden state is computed as:

$$\mathbf{h} = \mathbf{x} \times W_E$$

where $\mathbf{x} \in \mathbb{R}^{1\times 8}$ is the one-hot input vector for `brown`. The embedding matrix $W_E \in \mathbb{R}^{8\times 3}$ contains 8 rows (indexed $i = 0, \ldots, 7$, one per vocabulary word) and 3 columns (indexed $m = 1, 2, 3$, one per embedding dimension). Expanding the matrix product element-wise for a single embedding coordinate h_m yields:

$$h_m = x_0 \cdot W_{E_{0,m}} + x_1 \cdot W_{E_{1,m}} + x_2 \cdot W_{E_{2,m}} + \cdots + x_7 \cdot W_{E_{7,m}}$$

Each term multiplies the one-hot input value at vocabulary index i (x_0 for `The` , x_1 for `quick` , x_2 for `brown` , through x_7 for `dog`) by the corresponding weight in row i of W_E . Just as in Layer 2, the input values x_i are fixed constants from the one-hot encoding and $W_{E_{i,m}}$ is the variable of differentiation. Only the term containing $W_{E_{i,m}}$ survives differentiation:

$$\frac{\partial h_m}{\partial W_{E_{i,m}}} = x_i$$

Applying the chain rule connects this local derivative to the loss gradient $\frac{\partial L}{\partial h_m}$ computed in the previous section:

$$\frac{\partial L}{\partial W_{E_{i,m}}} = \frac{\partial L}{\partial h_m} \cdot \frac{\partial h_m}{\partial W_{E_{i,m}}} = \frac{\partial L}{\partial h_m} \cdot x_i$$

The input vector $\mathbf{x}$ is one-hot encoded, so $x_i = 0$ for every vocabulary word except `brown` at index `2` , where $x_2 = 1$. Evaluating each row of W_E reveals a sparse update pattern:

- **Row 0 (`The`, $i = 0$):** $\frac{\partial L}{\partial W_{E_{0,m}}} = 0 \cdot \frac{\partial L}{\partial h_m} = 0$. The gradient is zero, leaving row 0 untouched.
- **Row 1 (`quick`, $i = 1$):** $\frac{\partial L}{\partial W_{E_{1,m}}} = 0 \cdot \frac{\partial L}{\partial h_m} = 0$. The gradient is zero, leaving row 1 untouched.
- **Row 2 (`brown`, $i = 2$):** $\frac{\partial L}{\partial W_{E_{2,m}}} = 1 \cdot \frac{\partial L}{\partial h_m} = \frac{\partial L}{\partial h_m}$. The upstream gradient passes directly into row 2.
- **Rows 3 to 7 (`fox`, `jumps`, `over`, `lazy`, `dog`):** $\frac{\partial L}{\partial W_{E_{i,m}}} = 0 \cdot \frac{\partial L}{\partial h_m} = 0$. All remaining rows receive zero gradient.

This sparse gradient routing mirrors the row-extraction mechanism from the forward pass, operating in reverse. During the forward pass, multiplying by the one-hot vector extracted a single row from W_E. During backpropagation, multiplying by the one-hot input deposits the gradient into that same row. Only `brown`'s embedding vector receives an update from this training example; every other word in the vocabulary remains untouched.

The Weight Update

With the gradient computed end-to-end, the gradient descent weight update rule is applied:

$$W_{\text{new}} = W_{\text{old}} - \alpha \cdot \nabla L$$

The scalar α is the **learning rate**, which governs the step size of each parameter update. Applying an unscaled gradient to jump directly to a position that predicts `fox` would corrupt the representation because `brown` does not appear in a single context. Across the training corpus, `brown` precedes `fox`, `bear`, `sugar`, `eyes`, and `shoes`. Each of these contexts generates a different gradient, pulling `brown`'s vector in a different direction in the embedding space.

An oversized learning rate causes each training example to overwrite the previous update, forcing the vector for `brown` to oscillate chaotically instead of settling into a stable position. A small learning rate ensures that each individual example contributes a microscopic nudge. Across millions of iterations, these nudges accumulate statistically: directions that are consistently reinforced across many contexts grow dominant, whereas contradictory or noisy signals cancel out. The converged position of the vector reflects not any single sentence, but the **aggregate statistical geometry** of every context in which `brown` appeared.

Completing this single training step shifts the 3-dimensional vector for `brown` by a fraction of a decimal point. That single change is imperceptible in isolation, yet training executes this identical operation billions of times across every token in the corpus, adjusting every vector that participates in each forward pass. The cumulative effect of these microscopic nudges and the geometric structure that emerges from them form the subject of the next chapter.

Chapter 6: Convergent Geometry and Linear Substructures

Completing a single training step shifts the vector for `brown` by a fraction of a decimal point. One gradient, computed from one sentence, adjusts three coordinates by a microscopic amount. This update operation repeats across every sentence in the training corpus; every token that participates in a forward pass has its embedding vector nudged by the resulting gradient. A corpus of billions of words produces billions of gradient updates, each applying a tiny push in the embedding space. This chapter traces the cumulative effect of those pushes and the geometric structure that emerges from them.

From One Nudge to a Million

Chapters 4 and 5 derived the gradient for a single training example: the sequence `The` `quick` `brown` predicting `fox`. That gradient adjusted a single row of W_E (row `2`, corresponding to `brown`) in a direction that makes the network slightly more likely to predict `fox` in this context.

Across a large training corpus, `brown` does not appear solely before `fox`; it precedes thousands of distinct words:

- `brown` `fox`: The toy corpus example traced in Chapters 3 through 5.
- `brown` `bear`: A different animal noun sharing animate action contexts.
- `brown` `sugar`: A culinary ingredient appearing in food contexts.
- `brown` `eyes`: A physical trait appearing in descriptive contexts.
- `brown` `shoes`: An article of clothing appearing in apparel contexts.

Each of these contexts executes the pipeline from Chapters 4 and 5: a forward pass produces a prediction, the loss measures the error, and backpropagation computes a gradient $\frac{\partial L}{\partial \mathbf{h}} = (\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$ that nudges `brown`'s vector. Each context produces a *different* gradient because the target word changes and the error signal $\hat{\mathbf{y}} - \mathbf{y}$ points in a different direction.

The learning rate α prevents any single gradient from dominating. Each individual nudge is microscopic. Across millions of iterations, these nudges accumulate *statistically*: gradient components that are consistently reinforced across many different contexts grow dominant, whereas components that point in contradictory directions cancel out. The converged position of `brown`'s vector is determined not by any single sentence, but by the aggregate statistics of every context in which `brown` appeared.

This statistical accumulation realizes the distributional hypothesis introduced in Chapter 3. Chapters 3, 4, and 5 establish the mechanism by which a neural network discovers this principle automatically: gradient descent, applied at scale, forces words with similar contextual distributions into neighboring regions of the embedding space. No explicit definition of meaning is ever provided to the network; meaning emerges as a geometric side effect of next-token prediction.

Convergent Geometry

The distributional hypothesis explains both why a single word's vector stabilizes and why *different* words converge toward one another.

Consider two animal nouns from the toy vocabulary: `fox` and `dog`. Across a large corpus, both words frequently precede the same target words:

- **Shared Action Context:** `The` **`fox`** `jumps over the fence` / `The` **`dog`** `jumps over the fence`
- **Shared Observation Context:** `A wild` **`fox`** `appeared nearby` / `A wild` **`dog`** `appeared nearby`
- **Shared Passive Context:** `The` **`fox`** `was spotted nearby` / `The` **`dog`** `was spotted nearby`

This contextual overlap has a direct mathematical consequence. Tracing the gradient formula from Chapters 4 and 5 across two training examples that share `jumps` (index `4` in the toy vocabulary) as their target word demonstrates how shared contexts align gradient updates.

Example 1: Context Token `fox` Predicting `jumps`

The network executes the forward pass from Chapter 3 by extracting `fox`'s embedding from W_E, projecting through W_U, and applying Softmax to produce a predicted probability distribution. Suppose the network outputs:

$$\hat{\mathbf{y}}_{\text{fox}} = [0.05 \quad 0.05 \quad 0.05 \quad 0.10 \quad 0.30 \quad 0.20 \quad 0.10 \quad 0.15]$$

The network assigns 30% probability to `jumps` at index `4`. The ground truth $\mathbf{y}$ is a one-hot vector with a 1 at index `4` and 0 everywhere else. Computing the error signal $\hat{\mathbf{y}} - \mathbf{y}$ from Chapter 4 element by element yields:

$$\hat{\mathbf{y}}_{\text{fox}} - \mathbf{y} = [0.05 \quad 0.05 \quad 0.05 \quad 0.10 \quad \mathbf{-0.70} \quad 0.20 \quad 0.10 \quad 0.15]$$

Seven of the eight components are small positive numbers (the largest is 0.20 for `over`). The eighth component at index `4` (the target word `jumps`) evaluates to $0.30 - 1 = -0.70$, a large negative value whose magnitude is more than three times larger than any other entry.

Example 2: Context Token `dog` Predicting `jumps`

In a second sentence, `dog` serves as the input token. Distinct weights in `dog`'s embedding row cause the forward pass to produce a different probability distribution:

$$\hat{\mathbf{y}}_{\text{dog}} = [0.08 \quad 0.03 \quad 0.07 \quad 0.12 \quad 0.25 \quad 0.15 \quad 0.12 \quad 0.18]$$

Sharing the target word `jumps` means $\mathbf{y}$ remains the identical one-hot vector with 1 at index `4`:

$$\hat{\mathbf{y}}_{\text{dog}} - \mathbf{y} = [0.08 \quad 0.03 \quad 0.07 \quad 0.12 \quad \mathbf{-0.75} \quad 0.15 \quad 0.12 \quad 0.18]$$

The individual predictions differ, yet the structure of the error signal matches: seven small positive values and one dominant negative value at index `4`. The largest component sits at the same index in both examples because both sentences share the target word `jumps`.

From Error Signal to Gradient Direction

The gradient that updates each input token's embedding is $(\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$, where $W_U^T \in \mathbb{R}^{8\times 3}$ is the transpose of the 3×8 un-embedding matrix. This matrix multiplication computes a weighted sum of the rows of W_U^T, where the eight components of $\hat{\mathbf{y}} - \mathbf{y}$ serve as the scalar weights. Let $\mathbf{r}_j \in \mathbb{R}^{1\times 3}$ denote row j of W_U^T, representing vocabulary word j's un-embedding weights. Expanding the gradient for the `fox` example yields:

$$\frac{\partial L}{\partial \mathbf{h}_{\text{fox}}} = 0.05 \cdot \mathbf{r}_0 + 0.05 \cdot \mathbf{r}_1 + 0.05 \cdot \mathbf{r}_2 + 0.10 \cdot \mathbf{r}_3 - 0.70 \cdot \mathbf{r}_4 + 0.20 \cdot \mathbf{r}_5 + 0.10 \cdot \mathbf{r}_6 + 0.15 \cdot \mathbf{r}_7$$

This linear combination sums eight 3-dimensional vectors scaled by their error coefficients. The term $-0.70 \cdot \mathbf{r}_4$ dominates the sum: its coefficient has the largest magnitude by a factor of more than three, and it is the only negative term. The vector $\mathbf{r}_4$ (the un-embedding weights for `jumps`) therefore governs the resulting gradient direction.

Expanding the `dog` example produces the same structural dominance:

$$\frac{\partial L}{\partial \mathbf{h}_{\text{dog}}} = 0.08 \cdot \mathbf{r}_0 + 0.03 \cdot \mathbf{r}_1 + 0.07 \cdot \mathbf{r}_2 + 0.12 \cdot \mathbf{r}_3 - 0.75 \cdot \mathbf{r}_4 + 0.15 \cdot \mathbf{r}_5 + 0.12 \cdot \mathbf{r}_6 + 0.18 \cdot \mathbf{r}_7$$

The same row $\mathbf{r}_4$ dominates with a comparably large negative coefficient (-0.75 versus -0.70). Small positive contributions differ slightly between the two examples, yet the dominant pull along $\mathbf{r}_4$ (determined by the shared target `jumps`) aligns both gradients in nearly the same direction.

The words `fox` and `dog` share numerous target words across the corpus, including `jumps`, `runs`, `sleeps`, and `was`. Each shared target places the dominant component of the error signal at the same index, pulling both gradients along the same row of W_U^T. Across millions of training examples, these aligned pushes accumulate, drawing `fox`'s and `dog`'s vectors toward the same region of the embedding space. Conversely, words that rarely share targets (such as `fox` and `over`) receive error signals whose dominant components sit at different indices, producing divergent gradients that keep their vectors separated.

This process produces **semantic clustering**: after training, the embedding space is no longer the featureless, isotropic shell described in Chapter 2. Animals cluster near other animals, verbs cluster near other verbs, and color adjectives cluster near other color adjectives. None of this taxonomy was programmed into the network; it emerged from the co-occurrence statistics of the training data through the gradient descent updates derived in Chapters 4 and 5.

Measuring the Geometry

Evaluating claims such as "`fox` and `dog` converge in the embedding space" requires quantifying the geometric alignment between two vectors. The standard metric is **cosine similarity**, which measures the cosine of the angle between two vectors in d-dimensional space.

The Formula

Given two vectors $\mathbf{a}$ and $\mathbf{b}$, each with d dimensions, their **dot product** is defined as the sum of the element-wise products across corresponding coordinates:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{m=1}^{d} a_m \cdot b_m$$

The calculation also requires each vector's **magnitude** (its Euclidean norm), which measures the length of the vector in d-dimensional space:

$$\|\mathbf{a}\| = \sqrt{\sum_{m=1}^{d} a_m^2}$$

Cosine similarity divides the dot product by the product of the two magnitudes. This normalization removes the effect of vector length, isolating the directional alignment between the two vectors:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

The ratio produces a single scalar bounded between -1 and $+1$:

- **Aligned Vectors ($+1$):** Point in the identical direction, indicating maximum directional similarity.
- **Orthogonal Vectors (0):** Meet at a $90°$ angle, indicating zero directional relationship.
- **Opposite Vectors (-1):** Point in diametrically opposed directions, indicating maximum directional dissimilarity.

A Concrete Computation

To ground this formula in the 3-dimensional toy model, suppose billions of gradient updates have converged the vectors for `fox` and `dog` to the following coordinates:

$$\mathbf{v}_{\text{fox}} = [0.9 \quad -0.4 \quad 0.3], \quad \mathbf{v}_{\text{dog}} = [0.7 \quad -0.1 \quad 0.5]$$

1. **Dot Product:** Multiply each pair of corresponding dimensions and sum the products:

$$\mathbf{v}_{\text{fox}} \cdot \mathbf{v}_{\text{dog}} = (0.9)(0.7) + (-0.4)(-0.1) + (0.3)(0.5) = 0.63 + 0.04 + 0.15 = 0.82$$

2. **Euclidean Magnitudes:** Square each coordinate, sum the squares, and extract the square root:

$$\|\mathbf{v}_{\text{fox}}\| = \sqrt{0.9^2 + (-0.4)^2 + 0.3^2} = \sqrt{0.81 + 0.16 + 0.09} = \sqrt{1.06} \approx 1.030$$

$$\|\mathbf{v}_{\text{dog}}\| = \sqrt{0.7^2 + (-0.1)^2 + 0.5^2} = \sqrt{0.49 + 0.01 + 0.25} = \sqrt{0.75} \approx 0.866$$

3. **Cosine Similarity:** Divide the dot product by the product of the two magnitudes:

$$\cos(\mathbf{v}_{\text{fox}}, \mathbf{v}_{\text{dog}}) = \frac{0.82}{1.030 \times 0.866} = \frac{0.82}{0.892} \approx 0.92$$

A cosine similarity of 0.92 (near the maximum of $+1.0$) reflects the fact that both words are animal nouns appearing in strongly overlapping contexts. That alignment contrasts sharply with the initialization state derived in Chapter 2, where randomly initialized vectors in high-dimensional space start out mutually orthogonal with cosine similarities near 0.0 . Training transforms the isotropic initialization shell into a geometric manifold where directional proximity encodes semantic similarity.

Linear Substructures

Beyond semantic clustering, the emergent geometry of trained embeddings exhibits **linear substructures**, where vector addition and subtraction capture semantic and syntactic analogies.

A classic demonstration involves the words `king`, `queen`, `man`, and `woman`. Those four words lie outside the 8-word toy vocabulary, yet the geometric relationship they illustrate emerges in any embedding space trained on a sufficiently large corpus, where the following vector arithmetic holds to a close approximation:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

The difference $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}}$ defines a **displacement vector** pointing from `man`'s coordinates to `king`'s coordinates in the embedding space. Subtracting $\mathbf{v}_{\text{man}}$ cancels the shared masculine and human attributes while isolating the concept of royalty as a spatial direction and magnitude. Adding that same displacement vector to $\mathbf{v}_{\text{woman}}$ translates `woman`'s coordinates along the royalty axis, arriving near the converged position of `queen` in the embedding space.

This relational geometry extends across the vocabulary. Trained embedding spaces organize consistent linear substructures along numerous semantic and grammatical axes:

- **Gender Axis:** Encodes gender offsets where $\mathbf{v}_{\text{brother}} - \mathbf{v}_{\text{sister}} \approx \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{queen}}$.
- **Verb Tense Axis:** Encodes temporal inflections where $\mathbf{v}_{\text{walking}} - \mathbf{v}_{\text{walked}} \approx \mathbf{v}_{\text{swimming}} - \mathbf{v}_{\text{swam}}$.
- **Capital-Country Axis:** Encodes geopolitical relationships where $\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} \approx \mathbf{v}_{\text{Tokyo}} - \mathbf{v}_{\text{Japan}}$.

None of these linear relationships were explicitly programmed. No loss function instructed the network to encode gender as a linear direction or verb tense as a constant vector offset. These substructures emerged from the distributional statistics of the training corpus through the gradient descent process from Chapters 4 and 5, applying the $(\hat{\mathbf{y}} - \mathbf{y})$ error signal billions of times under the next-token prediction objective.

Chapter 7: The Embedding Tensor and the Limits of Static Representations

The previous chapter established the convergent geometry and linear substructures that emerge from gradient descent at scale. This final chapter examines how static embeddings operate when processing full token sequences rather than isolated words.

From Single-Token Lookup to the Sequence Tensor

Throughout Chapters 1 through 6, the network processed a single token at a time. The input was one one-hot row vector $\mathbf{x} \in \mathbb{R}^{1\times 8}$, and the output was one embedding vector $\mathbf{h} \in \mathbb{R}^{1\times 3}$ extracted from a single row of W_E. Production language models do not process words in isolation; they ingest entire token sequences simultaneously. Generalizing from single-token lookup to full-sequence processing is the final mechanical step before handing the representation off to the Transformer architecture.

Instead of feeding a single one-hot vector into W_E, the lookup stacks T one-hot vectors (one per token in the input sequence) into a matrix. The variable T denotes the **sequence length**, representing the number of tokens processed in parallel.

Consider the full toy sequence `The` `quick` `brown` `fox`, which has a sequence length of $T = 4$ tokens. Each token maps to a one-hot row vector in $\mathbb{R}^{1\times 8}$ (one column per vocabulary word). Constructing each one-hot vector individually places a 1 at index `0` for `The`, index `1` for `quick`, index `2` for `brown`, and index `3` for `fox`. Stacking all four vectors vertically produces the **one-hot input matrix**:

$$X_{\text{one-hot}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}_{4\times 8}$$

The first row holds the one-hot encoding for `The` (a 1 at index `0`, zeros elsewhere), the second row encodes `quick` (index `1`), the third row encodes `brown` (index `2`), and the fourth row encodes `fox` (index `3`). The resulting matrix has shape $T \times V = 4 \times 8$: one row per token in the sequence, and one column per word in the vocabulary.

Batched Row Extraction

Multiplying the one-hot input matrix by the embedding matrix $W_E \in \mathbb{R}^{8\times 3}$ from Chapter 3 projects the entire sequence into the continuous space:

$$X = X_{\text{one-hot}} \times W_E$$

Chapter 1 established that multiplying a single one-hot vector by W_E extracts the corresponding row because the vector-matrix product zeroes out every row except the index where the 1 appears. Matrix multiplication applies that identical extraction to each row of $X_{\text{one-hot}}$ independently, retrieving four rows from W_E in a single batched operation:

$$X = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ -0.3 & 0.9 & 0.4 \\ 0.2 & -0.2 & 0.1 \end{bmatrix}_{4\times 3}$$

Each row of X holds the 3 -dimensional embedding vector for the corresponding token in the sequence:

Sequence Row	Token	Vocabulary Index	Extracted Embedding Vector
Row 0	`The`	Index `0`	`[0.1, -0.4, 0.2]`
Row 1	`quick`	Index `1`	`[0.5, 0.1, -0.8]`
Row 2	`brown`	Index `2`	`[-0.3, 0.9, 0.4]`
Row 3	`fox`	Index `3`	`[0.2, -0.2, 0.1]`

Row `2` recovers the vector $\mathbf{h} = [-0.3 \quad 0.9 \quad 0.4]$ used throughout Chapters 1 and 3 through 5 as the hidden state for `brown`. Batching the sequence operation leaves the underlying lookup mechanism unchanged.

The resulting matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$ is the **embedding tensor** for the input sequence: a grid of T rows and d_{model} columns encoding the full sequence as a block of continuous vectors. In the toy model, X is a 4×3 matrix. In a larger model processing a 512-token sequence with $d_{\text{model}} = 512$, the tensor X forms a 512×512 matrix where each of the 512 rows holds a high-dimensional vector shaped by billions of gradient updates to encode the distributional signature of its token.

The Limits of Static Geometry

The embedding tensor X is dense, continuous, and semantically structured. Each row carries the distributional signature of its token, distilled from billions of training contexts into d_{model} dimensions. Static, context-independent lookup tables nevertheless impose two structural limitations that no amount of additional training can overcome.

Limitation 1: Context-Blindness (Polysemy)

Every occurrence of a word extracts the identical row of W_E, regardless of the surrounding sentence. The word `apple` in *"The chef sliced a crisp apple"* and `apple` in *"The engineer opened an apple laptop"* produce identical embedding vectors. The trained vector for `apple` settles into a geometric compromise at the center of mass of all its usages, blending fruit and consumer technology without isolating either meaning. That static compromise is the **superposition** bottleneck identified at the end of Chapter 3: a single fixed vector cannot represent multiple distinct meanings across shifting contexts.

Limitation 2: Order-Agnosticism (Positional Blindness)

The matrix multiplication $X_{\text{one-hot}} \times W_E$ treats each row of $X_{\text{one-hot}}$ independently without recording sequence order. The clauses `dog bites man` and `man bites dog` carry opposite meanings yet produce the identical set of three embedding vectors, merely permuted across the rows of X. The embedding matrix has no mechanism to encode word order because each row lookup depends solely on which vocabulary column contains the 1, not on the row's position within the sequence.

Handoff to the Transformer Architecture

These two limitations are not flaws in the gradient descent training process; they are inherent to any architecture that maps each vocabulary token to a single pre-computed vector. Resolving polysemy and encoding word order require a mechanism that inspects the entire sequence of embedding vectors and dynamically re-weights each token representation based on its neighbors.

The Transformer architecture provides that contextualization mechanism. The embedding tensor $X \in \mathbb{R}^{T \times d_{\text{model}}}$, carrying both its semantic geometry and its static limitations, serves as the direct input to the Transformer. Positional encodings first inject sequence-order information that the embedding lookup cannot provide, and the self-attention mechanism then allows each vector in the sequence to attend to every other vector, dynamically resolving ambiguity and context-dependence on a per-sentence basis. The static geometry derived across this series is not the final representation; it is the foundational coordinate space from which the Transformer begins its work.